

Phase 0 Discovery Document

Part B: Source Code Analysis & Refined Migration Plan

Comprehensive analysis of 634 Laravel PHP/Blade files, documenting every controller, middleware, validation rule, payment flow, reviewer workflow, and admin module. Includes corrections to Part A, 20 answered open questions, and 20 Migration Decision Records.

Document Version: Part B (v1.0)

Supersedes: Part A where corrections noted in Section 0

Prepared By: Principal Software Architect

Source: Laravel 11 + PHP 8.3 source code (634 files)

Lines Analyzed: ~25,000 lines of PHP + Blade

Date: July 2026



Table of Contents (Part B)

- 0. Corrections to Part A**
- 1. Controller Business Logic Inventory**
- 2. Validation Rules Catalog**
- 3. Middleware Stack Documentation**
- 4. Auth Flow Deep Dive**
- 5. Cart & Checkout Flow**
- 6. Payment Gateway Deep Dive**
- 7. Reviewer Workflow**
- 8. Admin Panel Flow**
- 9. Notification System**
- 10. Cron Jobs & Scheduled Tasks**
- 11. Helpers & Utilities**
- 12. Updated Schema Decision Record**
- 13. Updated NestJS Module Map**
- 14. 20 Open Questions — Answered**
- 15. Updated Risk Register (New Risks)**
- 16. Migration Decision Records (20 MDRs)**

Part B is best read alongside Part A. Part A covers: database analysis (67 tables), public surface audit, AWS architecture, 30-week roadmap, data migration plan, URL/SEO preservation, OWASP security checklist. Part B covers: business logic, validation, middleware, auth flows, payment flows, reviewer workflow, admin panel, notifications, cron, helpers, and 20 formal Migration Decision Records.

SECTION 0

Corrections to Part A

After receiving the Laravel source code, several assumptions in Part A were proven incorrect. This section catalogs all corrections. Part B supersedes Part A where conflicts exist. Part A remains as a historical snapshot of the initial analysis based on SQL dump + live site audit only.

0.1 Corrections Matrix

Topic	Part A Said	Part B Correction (Actual)
Product detail URL	/products/{slug}	/game-source-code/{slug} — confirmed in routes/web.php line 68. Laravel already has 301 redirects from /products/{slug} → /game-source-code/{slug} (lines 55-66).
Product reviews URL	/products/{slug}/reviews	/game-source-code/{slug}/reviews (route name: product.reviews)
Product comments URL	/products/{slug}/comments	/game-source-code/{slug}/comments (route name: product.comments)
Product changelog URL	/products/{slug}/changelog	/game-source-code/{slug}/changelog (route name: product.changelog)
Free products URL	Not mentioned	/free-products (route name: free.products) — separate page for is_free=1 items
Cookie policy URL	Not mentioned	/cookie-policy (route name: cookie.policy) + /cookie/accept endpoint
Sitemap	Mentioned but not detailed	Dynamic /sitemap.xml generated by SiteController@sitemap — includes homepage, products, blog posts, pages, policy pages
Placeholder image	Not mentioned	/placeholder-image/{WxH} — generates on-the-fly placeholder images via GD library
Language switch	Not mentioned	/change/{lang} — switches session language, redirects back
Maintenance mode URL	Not mentioned	/maintenance-mode route — bypasses middleware for status check
API layer assumption	Laravel has Next.js-ready API for is_web flag	WRONG. routes/api.php has only 4 endpoints: POST /api/verify-purchase-code, GET /api/author/products/all, GET /api/author/product/details. The is_web flag was likely for a mobile app or abandoned Next.js attempt. NestJS must build entire API from scratch.
Cart model	Single cart-per-user	Cart is per-item-per-product, not per-user. Each cart row = one product. No parent Cart entity. Cart ID matches product_id+user_id/session_id combo.
Payment gateways count	3 active (Razorpay, PayPal, manual UPI)	Confirmed 3 active. CheckoutController explicitly filters gateways to PayPal + Razorpay only (line 161). Manual UPI (Google Pay) shows as QR code, handled via manualDepositUpdate flow.
Auth flow	Single login endpoint	THREE separate auth surfaces with separate guards: /login (user), /admin (admin), /reviewer (reviewer). Each has own forgot-password + reset-password flow. Admin can impersonate users and reviewers.
Wallet payment	Not mentioned	Wallet payment path EXISTS in PaymentController::orderFromAccountBalance() — if user has sufficient balance, payment bypasses gateway entirely. Triggered when gateway='wallet'.

Topic	Part A Said	Part B Correction (Actual)
Author fee calculation	Static fee per category	DYNAMIC. Author fee = authorLevel.fee (percentage) * cartItem.price. AuthorLevel determined by user's total_sold_amount vs minimum_earning threshold. Synced after every sale.
Product status states	Listed 3 (pending/approved/permanent-down)	Actually 6 states: 0=PENDING, 1=APPROVED, 2=SOFT_REJECTED, 3=HARD_REJECTED, 4=DOWN (soft disabled), 5=PERMANENT_DOWN. Plus separate product_updated field with 5 states: 0=NO_UPDATE, 1=PENDING, 2=APPROVED, 3=SOFT_REJECT, 4=HARD_REJECT.
Reviewer assignment	Reviewer reviews any product in their subcategory	Reviewer subcategories stored as JSON array in reviewers.subcategories. Routes enforce: pendingProducts() filters where IN subcategories AND assigned_to IN [0, auth_id]. Reviewer must explicitly assign product to themselves before review.
Product file lifecycle	File uploaded by seller, immediately available	TWO-STAGE. Seller uploads → product.temp_file. Reviewer approves → temp_file moves to product.file (atomic swap). On hard reject/permanent down with no sales → file physically deleted.
Reviews in production DB	Assumed genuine user reviews	MANY ARE SYNTHETIC. routes/console.php contains Artisan command reviews:add-foreign that generates fake reviews from foreign users with phrases like 'Reviewed from Vietnam'. This explains the country-specific review patterns observed in SQL dump.
Total sales counts	Assumed genuine	ALSO SYNTHETIC. products:set-random-sales Artisan command sets random total_sold values (default 5-20) across all products. Migration must preserve these numbers (user-facing) but new system should disable these commands.
Cookie consent	Not detailed	Cookie consent via gdpr_cookie cookie, 43200 min (30 days) expiry. Cookie policy page configurable from admin. Cookie acceptance is AJAX via /cookie/accept endpoint.
Multi-language	Disabled, no infrastructure	Infrastructure EXISTS. LanguageMiddleware sets locale from session. Language model has is_default flag. Admin can manage languages via /admin/language. Currently only English configured but multi-language=1 in general_settings.
Referral system	Disabled, schema-only	Code EXISTS. Referral::processReferralCommission() invoked in PaymentController after every successful sale IF gs('referral') enabled AND user.ref_by set. Currently disabled (referral=0) but fully functional.
KYC system	Disabled, schema-only	Code EXISTS. KycMiddleware blocks withdrawal if user.kv != VERIFIED. KYC form collects data, admin approves via admin/users/kyc-approve endpoint. Currently disabled (kv=0) but fully functional.
2FA system	Disabled, schema-only	Code EXISTS. GoogleAuthenticator library integrated. verifyG2fa() helper validates TOTP codes. User can enable via /user/twofactor. Currently disabled (ts=0 for all users) but fully functional.

Topic	Part A Said	Part B Correction (Actual)
Author levels gamification	Forward-looking, not implemented	ACTIVE. AuthorLevel has minimum_earning threshold + fee percentage. After every sale, system queries all levels where minimum_earning <= author.total_sold_amount and syncs to author_level_user pivot. Author fee is dynamically calculated from highest applicable level.
Admin panel scope	Unknown, needed source code	MASSIVE. 508 lines of admin routes covering: dashboard, blog (categories+posts), categories, subcategories, review categories, authors, author levels, comments, reviews, products, reviewers, ads, coupons, sales, campaigns, users, subscribers, gateways (automatic+manual), deposits, withdrawals, reports, support tickets, languages, file storage (FTP/Wasabi/DO/Backblaze), general settings, socialite credentials, system config, logo/icon, custom CSS, sitemap, robots, cookie, maintenance mode, author info form, KYC settings, cron config, notification templates, email/SMS/push settings, extensions, system info, frontend page builder, SEO.
Reviewer panel scope	Unknown, needed source code	8 product queues (pending, awaiting, assigned, approved, soft-rejected, hard-rejected, down, permanent-down, updated) + assign, approve, reject (soft/hard), approve update, reject update, reply activity, download product, download temp file.
Cron jobs	DB-driven, unknown specifics	Two types: (1) is_default=1 cron jobs invoke controller methods directly (e.g., CronController::campaignExpired), (2) is_default=0 cron jobs hit external URLs via CurlRequest. Triggered via /cron endpoint OR admin manual run. Logs to cron_job_logs with duration + error.
Notification system	Basic email	MULTI-CHANNEL. notify() helper sends via email, SMS, AND push simultaneously (configurable per template). Templates stored in notification_templates with email_body/sms_body/push_body + shortcodes. 6 supported SMS gateways (Nexmo, Twilio, MessageBird, etc.). Push via Firebase.
File storage	Local disk only	MULTI-DRIVER. general_settings.file_server field supports: 1=local, 2=FTP, 3=Wasabi, 4=DigitalOcean Spaces, plus Backblaze. Admin configures credentials in /admin/storage/*. FileUploader class uses Laravel Filesystem abstraction — switching drivers is config-only.

SECTION 1

Controller Business Logic Inventory

Every controller method with its route, business logic summary, and side effects. Organized by controller class. This is the authoritative reference for NestJS service extraction.

1.1 Controller Method Inventory

Controller	Method	Route	Business Logic Summary
SiteController	index	GET /	Loads homepage. Stores ?reference= param in session for referral tracking. Renders 'Template::home' with sections + SEO.
SiteController	pages	GET /{slug}	Loads static page from pages table by slug + tempname. Renders 'Template::pages'.
SiteController	products	GET /products	Lists approved+active+non-free products. Filters: search (title LIKE, tags JSON contains, author username LIKE), min_price, max_price, category, sub_category, rating (avg_rating =), date_range (created_at >= now-X), sort_by (new/best_rated/best_selling). Paginated 12/page. Counts products per date range for sidebar.
SiteController	freeProducts	GET /free-products	Lists approved+active+free (is_free=1) products. Grouped by category. Paginated 16/page.
SiteController	productDetails	GET /game-source-code/{slug}	Loads product by slug with author + comment count. Aborts 404 if status is PERMANENT_DOWN or HARD_REJECTED, OR if not user's own product and not APPROVED. Increments ProductView counter (1 row per product per day). Sets SEO meta from product.meta_title/meta_description or falls back to title/description (155 char limit).
SiteController	productReviews	GET /game-source-code/{slug}/reviews	Lists reviews for product. Aborts 404 if product not approved+not own OR review count < general_settings.min_reviews. Supports ?review_id filter. Paginated.
SiteController	productComments	GET /game-source-code/{slug}/comments	Lists top-level comments (parent_id=0, review_id=0) with user + replies. Supports ?comment_id filter. Paginated.
SiteController	productChangelog	GET /game-source-code/{slug}/changelog	Lists changelog entries. Aborts 404 if product not approved, OR changelog disabled in settings, OR product has pending update, OR no changelogs exist.
SiteController	contact	GET /contact	Loads contact form. Prefills user data if logged in.
SiteController	contactSubmit	POST /contact	Validates name/email/subject/message. Verifies captcha. Creates SupportTicket (status=0 OPEN, priority=2 MEDIUM, random ticket number). Creates AdminNotification. Creates first SupportMessage. Redirects to ticket view.
SiteController	policyPages	GET /policy/{slug}	Loads policy page (Privacy/Terms/Refund) from frontends table where data_keys='policy_pages.element'.
SiteController	changeLanguage	GET /change/{lang}	Sets session lang. Falls back to 'en' if language not found. Redirects back.
SiteController	cookieAccept	GET /cookie/accept	Queues gdpr_cookie for 43200 minutes (30 days).
SiteController	cookiePolicy	GET /cookie-policy	Loads cookie policy from frontends table. Aborts 404 if cookie status disabled.
SiteController	placeholderImage	GET /placeholder-image/{WxH}	Generates placeholder image on-the-fly via PHP GD library. Outputs JPEG with dimensions text.

Controller	Method	Route	Business Logic Summary
SiteController	maintenance	GET /maintenance-mode	Shows maintenance page if maintenance_mode=1, else redirects home.
SiteController	incrementClick	POST /click/{id}	Increments advertisement click counter. Returns JSON.
SiteController	sitemap	GET /sitemap.xml	Generates dynamic XML sitemap with: homepage, products listing, free products, blog index, contact, cookie policy, all pages, all policy pages, all approved products, all published blog posts. Returns application/xml.
CartController	index	GET /cart	Lists cart items by user_id (auth) OR session_id (guest). Checks if any active coupon exists. Clears applied coupon from cart + session.
CartController	store	POST /cart	Validates license (1 or 2) + product_id. Checks if product already in cart (prevents duplicate). Calculates price: Personal license = product.price (minus campaign discount if active). Commercial license = product.price_cl. Adds buyer_fee. Stores addon service flags (reskin/publish/store_optimization). Returns JSON with cartQty for header counter.
CartController	delete	DELETE /cart/{id}	Deletes cart item by product_id (matches user_id or session_id). Returns JSON for AJAX or redirects for non-AJAX.
CartController	collectionToCart	GET /collections/{id}/add-to-cart	Adds all products from a collection to cart. Iterates collection.products and calls getDataForCart() for each.
CartController	toggleExtended	GET /cart/toggle-extended/{id}	Toggles is_extended flag on cart item. Updates extended_amount from product's twelveMonthExtendedFee(). Clears coupon.
CartController	toggleAdditionalService	POST /cart/toggle-additional-service/{id}	Toggles reskin_selected/publish_selected/store_optimization_selected flag. Clears coupon. Returns updated additional_services_price + total_price.
CartController	applyCoupon	GET /cart/apply-coupon	Validates coupon_code + subtotal + cart_ids. Checks: coupon exists, status=1, within date range, subtotal >= minimum_spend, total usage < usage_limit_per_coupon, user usage < usage_limit_per_user. Calculates discount (percent or fixed). Updates cart rows with coupon_id + discount. Stores coupon_total + coupon_id in session. Returns JSON.
CheckoutController	index	GET /checkout	If not auth: stores checkout_return_url in session. Loads cart items (user or session). Aborts if cart empty. Calculates subtotal = sum(price + buyer_fee + extended_amount + addon_services). Loads active gateways FILTERED to PayPal + Razorpay only (lines 60-82 of checkout).
CheckoutController	processCheckout	POST /checkout/process	Validates auth + cart non-empty + user not buying own products + profile complete. Returns JSON success or error.
CheckoutController	payment	GET /checkout/payment	Loads payment page. Re-validates auth + cart + own-product check. Loads only PayPal + Razorpay gateways via whereIn('code', ['paypal', 'razorpay']).

Controller	Method	Route	Business Logic Summary
CheckoutController	processPayment	POST /checkout/payment/process	Validates gateway. Re-validates auth + cart + own-product. Creates Order via createOrder(). Creates Deposit record with method_code, currency, amount, charge, final_amount. Calls gateway-specific ProcessController::process(). Returns redirect URL.
CheckoutController	thankYou	GET /checkout/thank-you	Loads order from session order_trx. Clears session. Renders thank-you page with order details + download links.
CheckoutController	createOrder (private)	(internal)	Creates Order row. For each cart item: determines author's AuthorLevel (highest minimum_earning), calculates seller_fee = level.fee% * price. Creates OrderItem with purchase_code (unique), license, extended_amount, addon flags, seller_earning = (price - (seller_fee + discount)) + extended_amount + addon_services_amount.
PaymentController	deposit	GET /deposit	Shows deposit methods page. Loads all active gateways.
PaymentController	depositInsert	POST /deposit/insert	Validates gateway. If gateway='wallet': check balance >= amount, call orderFromAccountBalance(). Else: find GatewayCurrency, validate min/max amount, calculate charge + final_amount, create Deposit, redirect to depositConfirm.
PaymentController	orderFromAccountBalance (private)	(internal)	Wallet payment path. Sets order.payment_status=SUCCESS. Deducts buyer.balance. Creates Transaction (type='-', remark='purchase'). For each OrderItem: calculates seller_earning, credits author.balance, increments author.total_sold, increments product.total_sold, deducts seller_fee, syncs author levels, creates author Transaction (type='+', remark='new_sale'). Deletes cart. Triggers referral commission if enabled.
PaymentController	appDepositConfirm	GET /app/deposit/confirm/{hash}	Mobile app deeplink. Decrypts hash to deposit ID. Logs in user. Sets Track session. Redirects to depositConfirm.
PaymentController	depositConfirm	GET /deposit/confirm	Reads Track session. Loads Deposit. If method_code >= 1000: redirect to manualDepositConfirm. Else: dynamically instantiate gateway ProcessController (e.g., App\Http\Controllers\Gateway\Razorpay\ProcessController), call process(), render returned view.

Controller	Method	Route	Business Logic Summary
PaymentController::userDataUpdate (static)	(called from gateway IPN)	THE BIG ONE. Called by every gateway IPN after successful payment. Sets deposit.status=SUCCESS. Credits user.balance. Creates payment Transaction (type='+', remark='payment'). If deposit has order_id: sets order.payment_status=SUCCESS, deducts user.balance, creates purchase Transaction (type='-', remark='purchase'), for each OrderItem: credits author.balance with seller_earning, deducts seller_fee, increments counters, syncs author levels, creates author Transactions. Deletes cart. Triggers referral. Sends DEPOSIT_COMPLETE notification.	
PaymentController	manualDepositConfirm	GET /deposit/manual	Loads manual payment confirmation page (for UPI/QR). Shows gateway form fields.
PaymentController	manualDepositUpdate	POST /deposit/manual	Validates form data (e.g., UPI transaction ID, screenshot). Sets deposit.status=PENDING. Creates AdminNotification. Sends DEPOSIT_REQUEST notification. Admin manually approves later.
LoginController	showLoginForm	GET /login	Renders login form. Calls Intended::identifyRoute() to remember intended URL.
LoginController	login	POST /login	Validates (username OR email) + password. Verifies captcha. Stores pre_login_session_id for cart transfer. Throttles login attempts. On success: calls authenticated(). On failure: redirects back (or to checkout if checkout_return_url set).
LoginController	findUsername	(internal)	Determines if input is email (FILTER_VALIDATE_EMAIL) or username. Merges into request.
LoginController	logout	GET /login/logout	Logs out, invalidates session, redirects to login.
LoginController	authenticated	(post-login hook)	Saves UserLogin (IP, geo via getIpInfo, browser/OS via osBrowser). Transfers session cart to user cart. Checks checkout_return_url first, else Intended::getRedirection(), else dashboard.
LoginController	transferCartItemsToUser (private)	(internal)	Reads pre_login_session_id. For each session cart item: if user already has same product, merge flags + delete session row; else update row to user_id.
RegisterController	showRegistrationForm	GET /register	Checks gs('registration'). If disabled: shows registration_disabled view. Else: renders register form.

Controller	Method	Route	Business Logic Summary
RegisterController	validator	(internal)	Validates firstname, lastname, email (unique), password (min 6, mixed case + numbers + symbols + uncompromised if secure_password enabled), captcha, agree (required if gs('agree') enabled).
RegisterController	register	POST /register	Checks registration enabled. Validates. Regenerates session token. Verifies captcha. Stores pre_register_session_id. Creates user (ref_by from session reference). Logs in. Calls registered().
RegisterController	create (private)	(internal)	Creates User: email (lowercase), firstname, lastname, password (Hash::make), ref_by (from session reference), kv/ev/sv (set based on settings — if setting disabled, mark as verified). Creates AdminNotification. Creates UserLogin.
RegisterController	checkUser	POST /check-user	AJAX endpoint. Checks if email/mobile/username already exists. Returns {data: bool, type, field}.
RegisterController	registered	(post-register hook)	Transfers cart. Saves UserLogin. Checks checkout_return_url. If profile incomplete: redirect to user.data. Else: Intended::getRedirection() or dashboard.
SocialiteController	socialLogin	GET /social-login/{provider}	Redirects to OAuth provider (Google/Facebook/LinkedIn). Uses SocialLogin library.
SocialiteController	callback	GET /social-login/callback/{provider}	Handles OAuth callback. Creates or logs in user. On error: redirects home with notification.
User\ProductController	selectCategory	GET /product/select-category	Shows category selection for new product upload. Only categories with active subcategories.
User\ProductController	upload	GET /product/upload	Shows upload form. Requires category + sub_category + is_free params. Loads dynamic form from Form model (subcategory_attributes).
User\ProductController	edit	GET /product/edit/{slug}	Shows edit form for seller's own product. Loads dynamic form.
User\ProductController	saveProduct	POST /product/upload/{id?}	Creates or updates product. Validates: title, description, category, price (required if not free), price_cl, preview_file (file type validation from category.preview_file_types), main_file (zip only), demo_url (URL), preview_video (URL), is_free, reskin_price, publish_price, store_optimization_price. Plus dynamic form validation. Sanitizes description + changelog with HTMLPurifier. Generates slug via productSlug(). Uploads main file to temp_file (sets product_updated=PENDING if editing). Uploads thumbnail/preview/screenshot/preview_file. Saves meta_title + meta_description. Creates Activity log if message provided.
User\ProductController	productActivities	GET /product/{id}/activities	Shows activity log for seller's own product. Aborts 404 if not own product.
User\ProductController	replyActivity	POST /product/{slug}/activities	Seller replies to reviewer. Creates Activity with user_id.
User\ProductController	commenting	GET /product/commenting/{slug}	Toggles comment_disable flag on seller's own product.

Controller	Method	Route	Business Logic Summary
User\ProductController	uploadScreenshot (private)	(internal)	Extracts ZIP file to /screenshots/{slug}/screenshots. Validates all files are PNG/JPG/JPEG, no subdirectories. Removes old screenshots if editing.
User\ProductController	uploadThumbnail (private)	(internal)	Uploads thumbnail via fileUploader() to /productThumbnail/{slug}/.
User\ProductController	uploadPreviewImage (private)	(internal)	Uploads preview image to /productPreview/{slug}/ AND /productInlinePreview/{slug}/ (two locations).
User\ProductController	uploadPreviewFile (private)	(internal)	If image: same as uploadPreviewImage. Else: uploads to /previewFile/{slug}/ via FileUploader, sets audio_temp_file.
User\ProductController	uploadMainFile (private)	(internal)	Uploads main ZIP to /productFile/{slug}/. If editing: sets product_updated=PENDING. Stores filename in temp_file (NOT file).
User\ProductController	uploadPreviewVideo (private)	(internal)	Uploads preview video to /previewVideo/{slug}/. Currently commented out in saveProduct — preview_video stored as URL string instead.
Reviewer\ProductController	pendingProducts	GET /reviewer/product/pending	Lists pending products WHERE subcategory IN reviewer.subcategories AND assigned_to IN [0, auth_id].
Reviewer\ProductController	awaitingProducts	GET /reviewer/product/awaiting	Lists products WHERE subcategory IN reviewer.subcategories AND (status=PENDING OR product_updated=PENDING) AND status NOT IN [PERMANENT_DOWN, UPDATE_HARD_REJECT, HARD_REJECTED].
Reviewer\ProductController	assignedProducts	GET /reviewer/product/assigned	Lists products assigned to current reviewer.
Reviewer\ProductController	approvedProducts	GET /reviewer/product/approved	Lists products approved by current reviewer.
Reviewer\ProductController	softRejectedProducts	GET /reviewer/product/soft-rejected	Lists soft-rejected products by current reviewer.
Reviewer\ProductController	hardRejectedProducts	GET /reviewer/product/hard-rejected	Lists hard-rejected products by current reviewer.
Reviewer\ProductController	downProducts	GET /reviewer/product/down	Lists soft-disabled products by current reviewer.
Reviewer\ProductController	permanentDownProducts	GET /reviewer/product/permanent-down	Lists permanently-disabled products by current reviewer.
Reviewer\ProductController	updatedProducts	GET /reviewer/product/updated	Lists products with pending updates from current reviewer.
Reviewer\ProductController	assignProduct	POST /reviewer/product/{slug}/assign	Assigns product to current reviewer. Sets assigned_to=auth_id. Product must be unassigned + pending.
Reviewer\ProductController	details	GET /reviewer/product/{id}/details	Shows product details. Aborts 404 if assigned to different reviewer. Shows activity log.

Controller	Method	Route	Business Logic Summary
Reviewer\ProductController	approveProduct	POST /reviewer/product/{id}/approve	Approves product. Sets approved_by, published_at=now. ATOMIC FILE SWAP: temp_file → file, temp_file=null. Sets status=APPROVED. Creates Activity with '[Approve]' prefix. Sends PRODUCT_APPROVED email if author opted in.
Reviewer\ProductController	rejectProduct	POST /reviewer/product/{id}/reject/{type}	Rejects product with type (2=soft, 3=hard, 4=down, 5=permanent_down). Sets status=type. Creates Activity with appropriate prefix. If hard_reject or permanent_down AND no sales: physically deletes product file. Sends corresponding email notification.
Reviewer\ProductController	approveUpdate	POST /reviewer/product/update/{id}/approve	Approves product update. ATOMIC FILE SWAP: old file deleted, temp_file → file. Sets product_updated=NO_UPDATE, status=APPROVED, last_updated=now. Creates Activity with '[Update Approved]' prefix. Sends PRODUCT_UPDATE_APPROVED email.
Reviewer\ProductController	rejectUpdate	POST /reviewer/product/update/{id}/reject/{type}	Rejects update with type (3=soft, 4=hard). Sets product_updated=type. Deletes temp_file. Creates Activity with '[Update Soft/Hard Reject]' prefix. Sends PRODUCT_UPDATE_REJECTED email.
Reviewer\ProductController	downloadProduct	GET /reviewer/product/{id}/download	Downloads current product file (for review).
Reviewer\ProductController	downloadProductTemp	GET /reviewer/product/{id}/download-temp	Downloads temp_file (updated version pending review).
Reviewer\ProductController	replyActivity	POST /reviewer/product/{id}/activities	Reviewer replies to seller. Creates Activity with reviewer_id.
Admin\AdminController	dashboard	GET /admin/dashboard	Massive dashboard widget. Counts: total/verified/email-unverified/mobile-unverified users, total/approved/pending/soft-rejected/hard-rejected products, total sale + amount + seller charge + buyer charge + refunded, deposit totals, withdrawal totals, top 10 selling products. User login analytics (browser, OS, country) for last 30 days.
Admin\AdminController	getTopSellingItems	GET /admin/top-selling-items	AJAX endpoint. Returns top 10 products by total_sold. Supports time period filter (1=month, 6=months, 1=year, current month).
Admin\AdminController	getSalesData	GET /admin/sales-chart-data	AJAX endpoint. Returns chart data (date + total_sales + total_quantity). Supports time period filter (today, 7d, 15d, 1mo, 6mo, 1yr).
Admin\AdminController	depositAndWithdrawReport	GET /admin/chart/deposit-withdraw	AJAX endpoint. Returns chart data grouped by days (if range <= 30 days) or months (if range > 30 days).
Admin\AdminController	transactionReport	GET /admin/chart/transaction	AJAX endpoint. Returns +/- transactions chart data.
Admin\AdminController	notifications	GET /admin/notifications	Lists admin notifications with pagination.
Admin\AdminController	readAllNotification	POST /admin/notifications/read-all	Marks all admin notifications as read.

Controller	Method	Route	Business Logic Summary
Admin\AdminController	deleteAllNotification	POST /admin/notifications/delete-all	Truncates admin_notifications table.
Admin\AdminController	requestReport	GET /admin/request-report	Fetches bug/feature reports from external viserlab.com license server.
Admin\AdminController	reportSubmit	POST /admin/request-report	Submits bug/feature report to viserlab.com.
Admin\AdminController	downloadAttachment	GET /admin/download-attachments/{hash}	Downloads admin-uploaded attachment. Decrypts file path.
CronController	cron	GET /cron	THE MASTER CRON. Updates general_settings.last_cron. Loads all due cron_jobs (next_run < now AND is_running=YES). For each: creates CronJobLog, executes (controller method if is_default else cURL external URL), updates last_run + next_run, logs duration + error. Supports ?alias= for specific job, ?target=all for all.
CronController	campaignExpired	(called by cron)	Finds campaigns where end_date < now AND campaign_end=NO. Sets campaign_end=YES, status=EXPIRED. Updates all campaign_products to EXPIRED.
User\AuthorController	showProfile	GET /{username}	Shows public author profile. Includes portfolio, followers, following, collections.
User\AuthorController	follow	POST /author/follow/{user}	Toggles follow relationship. Updates total_follower/total_following counters.
User\AuthorController	earning	GET /author/earning	Seller earnings dashboard. Shows transactions + sales chart.
User\AuthorController	sells	GET /author/sales	Seller sales list with details.
User\AuthorController	refunds	GET /author/refunds	Seller refund requests list.
User\AuthorController	acceptRefundRequest	POST /author/refunds/{id}/accept	Seller accepts refund. Credits buyer, deducts seller, marks refund as accepted.
User\AuthorController	rejectRefundRequest	POST /author/refunds/{id}/reject	Seller rejects refund. Creates refund_activity log.
User\AuthorController	downloadProduct	GET /author/download-product/{purchaseCode}	Buyer downloads purchased product. Validates purchase_code ownership. Tracks in downloads table.
User\AuthorController	downloadFreeProduct	GET /author/download-free-product/{slug}	Downloads free product (no purchase required).
User\AuthorController	refundRequest	POST /author/refund/request/{purchase_code}	Buyer initiates refund request. Creates refund_request with reason.
User\AuthorController	saveComment	POST /author/comments/{productId}	Posts comment on product. Supports parent_id for replies.
User\AuthorController	saveReview	POST /author/reviews/{productId}	Posts review on product (requires purchase). Updates product.total_review + avg_rating. Updates author.total_review + avg_rating.

Controller	Method	Route	Business Logic Summary
User\AuthorController	reportComment	POST /author/comments/{id}/report	Reports comment for moderation. Creates reported_review with reason.
User\AuthorController	reportReview	POST /author/review/report	Reports review for moderation.
User\AuthorController	favorites	GET /author/favorites	Lists user's favorite products.
User\AuthorController	toggleFavorite	POST /author/favorites	Adds/removes product from favorites (product_user pivot).
User\AuthorController	collections	GET /author/collections	Lists user's product collections.
User\AuthorController	storeCollection	POST /author/collections	Creates new collection.
User\AuthorController	license	GET /author/license/index	License verification page.
User\AuthorController	verify	GET /author/license/verify	Verifies a purchase code (for buyers to validate license).
User\AuthorController	campaign	GET /author/campaign/index	Seller's campaign submissions.
User\AuthorController	referralList	GET /author/referral	Lists user's referrals.
TicketController	supportTicket	GET /ticket	Lists user's support tickets.
TicketController	openSupportTicket	GET /ticket/new	Shows new ticket form.
TicketController	storeSupportTicket	POST /ticket/create	Creates support ticket + first message. Validates subject + message.
TicketController	viewTicket	GET /ticket/view/{ticket}	Shows ticket thread with messages + attachments.
TicketController	replyTicket	POST /ticket/reply/{id}	Adds reply to ticket. Updates last_reply. Supports file attachments.
TicketController	closeTicket	POST /ticket/close/{id}	Closes ticket. Sets status=CLOSED.
TicketController	ticketDownload	GET /ticket/download/{attachment_id}	Downloads ticket attachment.
BlogController	index	GET /blog	Lists published blog posts. Paginated. Filterable by category.
BlogController	show	GET /blog/{slug}	Shows single blog post. Loads by slug + is_published=1.
Api\PurchaseController	verifyPurchasedCode	POST /api/verify-purchase-code	Public API. Validates purchase code, returns product info + license details. Used by external license verification tools.
Api\AuthorController	productsAll	GET /api/author/products/all	Returns all products by author (requires API key).
Api\AuthorController	productsDetail	GET /api/author/product/details	Returns product detail (requires API key).

SECTION 2

Validation Rules Catalog

All validation rules extracted from controller methods (inline `$request->validate()` calls) and `FormRequest` classes. NestJS migration should replicate these as Zod schemas in DTOs. Note: most validation is inline (not `FormRequest`) — this is a code quality issue in the original Laravel app that the NestJS migration should fix by centralizing validation in DTOs.

2.1 Validation Rules by Endpoint

Endpoint	Field	Rule	Notes
POST /register	firstname	required	
POST /register	lastname	required	
POST /register	email	required string email unique:users	Lowercased before save
POST /register	password	required confirmed min:6	If gs('secure_password'): mixedCase+numbers+symbols+uncompromised
POST /register	captcha	sometimes required	If captcha enabled
POST /register	agree	nullable OR required	Required if gs('agree') enabled
POST /login	username	required string	Email OR username — auto-detected via filter_var
POST /login	password	required string	
POST /contact	name	required	
POST /contact	email	required	No email format validation! (potential bug)
POST /contact	subject	required string max:255	
POST /contact	message	required	
POST /cart	license	required in:1,2	1=Personal, 2=Commercial
POST /cart	product_id	required exists:products,id	Must be approved product
GET /cart/apply-coupon	coupon_code	required string max:40	
GET /cart/apply-coupon	subtotal	required numeric gt:0	
GET /cart/apply-coupon	cart_ids	required	Array of cart item IDs
POST /checkout/payment/process	gateway	required	Method code — must be PayPal or Razorpay
POST /deposit/manual	(form data)	Dynamic via FormProcessor	Validated against gateway form_data JSON
POST /product/upload/{id?}	title	required	
POST /product/upload/{id?}	description	required	Sanitized with HTMLPurifier after validation
POST /product/upload/{id?}	category	required	Must be active
POST /product/upload/{id?}	sub_category	required	Must be active + belong to category
POST /product/upload/{id?}	price	required numeric gt:0	Nullable if is_free=1
POST /product/upload/{id?}	price_cl	required numeric gt:0	Commercial price — nullable if is_free=1

Endpoint	Field	Rule	Notes
POST /product/upload/{id?}	preview_file	nullable file mimes:{category.preview_file_types} max:{category.file_size}MB	Required if category.file_type=audio AND new product
POST /product/upload/{id?}	main_file	nullable file mimes:zip	Required for new product
POST /product/upload/{id?}	demo_url	required url	Nullable if category.file_type=audio
POST /product/upload/{id?}	preview_video	nullable url max:255	YouTube URL typically
POST /product/upload/{id?}	is_free	nullable in:0,1	Requires gs('free_item') enabled
POST /product/upload/{id?}	reskin_price	nullable numeric min:0	
POST /product/upload/{id?}	publish_price	nullable numeric min:0	
POST /product/upload/{id?}	store_optimization_price	nullable numeric min:0	
POST /product/upload/{id?}	meta_title	nullable string	strip_tags applied
POST /product/upload/{id?}	meta_description	nullable string	strip_tags applied
POST /reviewer/product/{id}/reject/{type}	reason	required (if type=soft_rejected OR down)	Nullable if type=hard_rejected OR permanent_down
POST /reviewer/product/update/{id}/reject/{type}	reason	required	
POST /admin/profile	name	required	
POST /admin/profile	email	required email	
POST /admin/profile	image	nullable image mimes:jpg,jpeg,png	Via FileTypeValidate rule
POST /admin/password	old_password	required	Hash::check against current
POST /admin/password	password	required min:5 confirmed	
POST /ticket/reply/{id}	message	required	
POST /ticket/create	subject	required	
POST /ticket/create	message	required	
POST /product/{slug}/activities	message	required	
POST /reviewer/product/{id}/activities	message	required	

SECTION 3

Middleware Stack Documentation

14 middleware classes enforce cross-cutting concerns: maintenance mode, language, template, authentication, authorization (3 roles), KYC, author status, profile completion, CSRF, demo mode. NestJS migration should replicate via guards, interceptors, and middleware. Note: VerifyCsrfToken excludes `/ipn/*` routes (payment webhooks) — NestJS should use raw body verification for webhooks instead.

3.1 Middleware Inventory

Middleware	Trigger	Behavior	NestJS Equivalent
MaintenanceMode	Global (web)	If <code>gs('maintenance_mode')=ENABLED</code> : redirect to <code>/maintenance-mode</code> (web) or return JSON error (api). Bypassed for <code>/maintenance-mode</code> route itself.	MaintenanceGuard — checks Redis flag, returns 503 JSON or redirects
LanguageMiddleware	Global (web)	Sets session lang from session or default Language model. Sets app locale.	I18nMiddleware — reads Accept-Language or cookie, sets req.locale
ActiveTemplateMiddleware	Global (web)	Shares activeTemplate + activeTemplateTrue with all views. Registers Template namespace. Adds pages to header/footer view composer.	Not needed — Next.js handles templates via component selection
Authenticate	Route-specific	Laravel default. Redirects to login if not authenticated.	JwtAuthGuard — returns 401 if no valid JWT
RedirectIfAuthenticated	Guest routes (<code>/login</code> , <code>/register</code>)	Redirects to dashboard if already logged in.	GuestGuard — redirects to <code>/dashboard</code>
CheckStatus	Authenticated routes	If user has <code>status=0</code> (banned) OR <code>ev=0</code> (email unverified) OR <code>sv=0</code> (mobile unverified) OR <code>tv=0</code> (2FA unverified): redirect to <code>/authorization</code> . API returns JSON error.	AccountVerifiedGuard — checks <code>user.emailVerifiedAt</code> + 2FA + status
RegistrationStep	Post-auth, pre-dashboard	If <code>user.profile_complete=0</code> : redirect to <code>/user-data</code> (profile completion form). API returns JSON error.	ProfileCompleteGuard — checks <code>user.profileCompleted</code> flag
KycMiddleware	Withdrawal routes	If <code>user.kv=UNVERIFIED</code> : redirect to KYC form. If <code>user.kv=PENDING</code> : redirect to dashboard. API returns JSON error.	KycGuard — checks <code>user.kycStatus</code> enum
AuthorMiddleware	Seller-only routes	If <code>user.is_author=0</code> : redirect to <code>/author-form</code> (author application).	AuthorGuard — checks <code>user.isAuthor</code> flag
RedirectIfAdmin	Admin guest routes	Redirects to <code>/admin/dashboard</code> if already admin-authed.	AdminGuestGuard
RedirectIfNotAdmin	Admin routes	Redirects to <code>/admin login</code> if not admin-authed.	AdminAuthGuard
RedirectIfReviewer	Reviewer guest routes	Redirects to <code>/reviewer/dashboard</code> if already reviewer-authed.	ReviewerGuestGuard
RedirectIfNotReviewer	Reviewer routes	Redirects to <code>/reviewer login</code> if not reviewer-authed.	ReviewerAuthGuard
RegistrationStep	(duplicate — listed above)		

Middleware	Trigger	Behavior	NestJS Equivalent
VerifyCsrfToken	All non-GET routes	Laravel default. Validates CSRF token. Excludes /ipn/* routes (payment webhooks).	CSRF middleware + SameSite cookies
PreventRequestsDuringMaintenance	Laravel default	Laravel built-in maintenance middleware.	N/A
CheckForMaintenanceMode	Laravel default	Laravel built-in maintenance middleware.	N/A
Demo	Route-specific	Restricts certain actions in demo mode (e.g., preventing admin password changes).	DemoGuard — environment-based feature flag

SECTION 4

Auth Flow Deep Dive

Three-guard architecture (user, admin, reviewer) with cart transfer, email/mobile/2FA verification, KYC, OAuth (Google), password reset, and admin impersonation. This section documents every auth flow end-to-end for faithful NestJS replication.

Three-Guard Architecture

Laravel uses three separate authentication guards configured in `config/auth.php`. Each guard has its own provider (Eloquent model), session driver, and password broker. The 'web' guard authenticates users from the users table. The 'admin' guard authenticates admins from the admins table. The 'reviewer' guard authenticates reviewers from the reviewers table. Sessions are isolated — logging in as admin does NOT log in as user, and vice versa. NestJS migration must replicate this with three separate JWT strategies: `UserJwtStrategy`, `AdminJwtStrategy`, `ReviewerJwtStrategy`. Each issues JWTs signed with a different secret (or different audience claim) to prevent cross-role token reuse.

Login Flow (User)

User visits `/login` → `LoginController@showLoginForm`. Submits credentials → `LoginController@login`. Controller calls `findUsername()` to detect email vs username. Validates (required|string) for username field, (required|string) for password. Verifies captcha via `verifyCaptcha()` helper. Stores `pre_login_session_id` in session for cart transfer. Throttles login attempts via `hasTooManyLoginAttempts()` (5 attempts per minute by default). On success: `attemptLogin()` fires, `authenticated()` hook runs. `authenticated()` does: (1) saves `UserLogin` record with IP/geo/browser/OS, (2) calls `transferCartItemsToUser()` to merge guest cart into user cart, (3) checks `checkout_return_url` session — if set, redirects there, (4) else calls `Intended::getRedirection()` for intended URL, (5) else redirects to `/dashboard`.

Registration Flow

User visits `/register` → `RegisterController@showRegistrationForm`. If `gs('registration')=0`: shows `registration_disabled` view. Submits form → `RegisterController@register`. Validates: `firstname` (required), `lastname` (required), `email` (required|email|unique:users), `password` (required|confirmed|min:6, with `mixedCase/numbers/symbols/uncompromised` rules if `gs('secure_password')=1`), `captcha` (sometimes|required), `agree` (required if `gs('agree')=1`). Regenerates session token. Verifies captcha. Stores `pre_register_session_id`. Creates User via `create()` method: lowercases email, `Hash::make(password)`, sets `ref_by` from session 'reference' (referral system), sets `kv/ev/sv` based on whether KYC/email/mobile verification is enabled (if disabled, auto-marks as verified). Creates `AdminNotification`. Creates `UserLogin`. Fires `Registered` event. Logs in user. Calls `registered()` hook which: transfers cart, checks `checkout_return_url`, redirects to `/user-data` if profile incomplete, else dashboard.

Email Verification

If `gs('ev')=1` (email verification enabled): new user has `ev=0`. After login, `CheckStatus` middleware detects `ev=0` and redirects to `/authorization`. `AuthorizationController@authorizeForm` shows verification form. User requests code via `/resend-verify/email` → `sendVerifyCode()` generates 6-digit code, stores in `user.ver_code`, stores timestamp in `user.ver_code_send_at`, sends email. User submits code via `/verify-email` → `emailVerification()` checks code + expiry (10 min default), sets `ev=1`.

Mobile Verification

Same as email but for mobile (`sv` field). Uses 6-digit SMS code. Requires SMS gateway configured (Twilio/Nexmo/etc. in `general_settings.sms_config`). Currently disabled (`sv=0` in settings).

2FA (Google Authenticator)

User enables via `/user/twofactor` → `create2fa()` generates secret, stores in `user.tsc`, shows QR code. User scans with Google Authenticator app, enters code → `verifyG2fa()` validates TOTP, sets `ts=1` (enabled). On subsequent logins: `authenticated()` sets `tv=0` (2FA unverified). `CheckStatus` middleware detects `ts=1` AND `tv=0`, redirects to `/authorization`. User enters TOTP code → `g2faVerification()` validates, sets `tv=1`.

KYC Verification

Required for withdrawals (`KycMiddleware`). User visits `/user/kyc-form` → `kycForm()` shows KYC form (configurable fields via admin). Submits documents → `kycSubmit()` stores in `user.kyc_data` JSON, sets `kv=2` (PENDING). Admin reviews via `/admin/users/kyc-data/{id}` → approves (`kv=1`) or rejects (`kv=0` + `kyc_rejection_reason`). Currently disabled (`kv=0` in settings) but code is fully functional.

Password Reset Flow

User visits `/password/reset` → `ForgotPasswordController@sendResetCodeEmail`. Generates 6-digit code, stores in `password_resets` table (or `admin_password_resets` / `reviewer_password_resets` depending on guard), emails code. User visits `/password/code-verify` → enters code → `verifyCode()` validates. User visits `/password/reset/{token}` → `ResetPasswordController@showResetForm`. Submits new password → `reset()` updates user password.

OAuth (Socialite)

User clicks 'Login with Google' on `/login` → `SocialiteController@socialLogin('google')` → `SocialLogin` library redirects to Google. Google redirects back to `/social-login/callback/google` → `SocialiteController@callback` → `SocialLogin::login()`. Library: (1) fetches Google user, (2) checks if user with that `google_id` exists, (3) if yes: logs in, (4) if no: creates new user with google data, logs in. Facebook and LinkedIn configured but disabled (`status=0` in `socialite_credentials`).

Impersonation

Admin can 'login as user' via `/admin/users/login/{id}` → `ManageUsersController@login()`. Admin can 'login as reviewer' via `/admin/reviewers/login/{id}` → `ManageReviewerController@loginAsReviewer()`. These endpoints log in the target user/reviewer using Laravel's `Auth::login()` method. No separate impersonation token — admin session is replaced. To return, admin must log out and log back in as admin. NestJS migration should implement this with a proper impersonation token (e.g., JWT with `impersonated_by` claim) for better audit trail and easier return-to-admin flow.

Cart Transfer on Login/Register

Critical UX feature: guest cart items are NOT lost when user logs in or registers. Before login: `pre_login_session_id` stored in session. After login: `transferCartItemsToUser()` iterates session cart items. For each: if user already has same product in cart, merge addon flags (`reskin/publish/store_optimization/extended`) and delete session row; else update row to set `user_id`, null `session_id`. Same flow on registration. NestJS migration must replicate this — store guest cart in Redis keyed by session ID, merge into user cart on auth success.

SECTION 5

Cart & Checkout Flow

Cart is intentionally per-item (no parent Cart entity). Checkout is 6-step flow with cart transfer, coupon, order creation, gateway routing. Two code paths for order creation (CheckoutController vs PaymentController) — must be unified in NestJS.

Cart Data Model

Cart is NOT a parent entity with child items — each cart row IS one item. Schema: id, product_id, user_id (nullable for guests), session_id (for guests), title, category_id, category, license (1=Personal, 2=Commercial), is_extended, extended_amount, price, seller_fee, buyer_fee, quantity, coupon_id, discount, reskin_selected, publish_selected, store_optimization_selected, timestamps. A single user can have multiple cart rows (one per product). Same product cannot be in cart twice — `getDataForCart()` checks for existing row by product_id + user_id/session_id and returns false if found.

Price Calculation

Personal license: $\text{price} = \text{product.price}$. If active campaign: $\text{price} = \text{campaign_product_price}[1] - \text{personalBuyerFee}$ (campaign price minus personal buyer fee). Commercial license: $\text{price} = \text{product.price_cl}$. If active campaign: $\text{price} = \text{campaign_product_commercial_price}[1] - \text{commercialBuyerFee}$. Buyer fee added separately: $\text{personalBuyerFee} = \text{category.personal_buyer_fee}$, $\text{commercialBuyerFee} = \text{category.commercial_buyer_fee}$. Extended license (12-month): $\text{extended_amount} = \text{category.twelve_month_extended_fee}$ (only if $\text{is_extended}=1$). Addon services: $\text{reskin_price} + \text{publish_price} + \text{store_optimization_price}$ from product fields (only if respective flag = 1).

Coupon System

Coupon applied via GET `/cart/apply-coupon?coupon_code=X&subtotal=Y&cart_ids=[1,2,3]`. Validates: coupon exists, $\text{status}=1$, $\text{start_date} \leq \text{now} \leq \text{end_date}$, $\text{subtotal} \geq \text{minimum_spend}$, $\text{total usage count} < \text{usage_limit_per_coupon}$ (if set), $\text{user's usage count} < \text{usage_limit_per_user}$ (if set). Discount calculation: if $\text{discount_type}=2$ (percent): $\text{discount} = (\text{price} + \text{buyer_fee} + \text{extended_amount}) * \text{amount} / 100$. If $\text{discount_type}=1$ (fixed): $\text{discount} = \text{amount}$. Updates each cart row with $\text{coupon_id} + \text{discount}$. Stores $\text{coupon_total} + \text{coupon_id}$ in session. Any cart modification (toggle extended, toggle addon, remove item) clears the coupon.

Checkout Flow (6 Steps)

Step 1: User visits /checkout → CheckoutController@index. If guest: stores checkout_return_url in session, shows login/register prompt. If auth: loads cart items, calculates subtotal + total, loads active PayPal + Razorpay gateways (hardcoded filter — manual UPI shown separately). Step 2: User clicks 'Place Order' → processCheckout() validates auth + cart + own-product check + profile complete. Step 3: User selects gateway → POST /checkout/payment/process → processPayment() validates gateway, creates Order via createOrder(), creates Deposit, calls gateway ProcessController::process(). Step 4: User completes payment on gateway (Razorpay checkout modal OR PayPal redirect). Step 5: Gateway redirects back to /deposit/confirm OR sends webhook to /ipn/{gateway}. Step 6: PaymentController::userDataUpdate() credits balances, creates transactions, clears cart, redirects to /checkout/thank-you.

Order Creation (Atomic)

CheckoutController::createOrder() (duplicated in PaymentController::createOrder() — code smell, should be extracted to service). Calculates: amount = sum(cart.price) - coupon_discount + sum(extended_amount) + sum(buyer_fee) + sum(addon_services_amount). Creates Order row with trx (unique 12-char). For each cart item: determines author's highest AuthorLevel (largest minimum_earning <= author.total_sold_amount), calculates seller_fee = level.fee% * price. Creates OrderItem with: purchase_code (unique format: {user_id}-{product_id}-{random}-{timestamp}), product_price, buyer_fee, seller_fee, license, extended_amount, addon flags, discount (per-item), seller_earning = (price - (seller_fee + discount)) + extended_amount + addon_amount. NOTE: seller_earning formula here is DIFFERENT from the one used in userDataUpdate() — this is a BUG in the original code (creates order item with one formula, then recalculates on payment success with a different formula). Migration must pick ONE formula and document it.

Wallet Payment Path

If user has sufficient balance (user.balance >= order.amount), they can pay via wallet — gateway='wallet'. PaymentController::orderFromAccountBalance() bypasses gateway entirely: immediately sets order.payment_status=SUCCESS, deducts buyer.balance, creates buyer Transaction, credits each author, creates author Transactions, clears cart, triggers referral. This is a no-fee payment path (no gateway charge). Currently UI does not expose this option (CheckoutController filters to PayPal+Razorpay only), but the code path exists. Migration should expose wallet payment as a third option for users with balance.

SECTION 6

Payment Gateway Deep Dive

Three active payment paths: Razorpay (INR, checkout modal + webhook), PayPal (USD, redirect + deprecated IPN), Manual UPI (QR + admin approval). Plus wallet payment path (code exists, UI doesn't expose). Webhook security analysis included.

Active Gateways Summary

Three payment paths are active in production: (1) Razorpay for INR payments — checkout modal embedded in page, webhook-based confirmation. (2) PayPal for USD payments — redirect to paypal.com, IPN-based confirmation. (3) Manual UPI (Google Pay QR) — buyer scans QR, sends screenshot, admin manually approves via `/admin/payment/approve/{id}`. The CheckoutController hardcodes a filter to only show PayPal + Razorpay in the gateway selector (line 161: `whereIn('code', ['paypal', 'razorpay'])`). Manual UPI is shown as a separate option in the checkout template directly.

Razorpay Flow (Step-by-Step)

1. User selects Razorpay in checkout → POST `/checkout/payment/process` with `gateway=110`.
2. CheckoutController creates Order + Deposit, calls `Razorpay\ProcessController::process()`.
3. `process()` reads gateway credentials from `gateway_parameters` JSON (`key_id`, `key_secret` — currently `rzp_live_SEgBGBF6PO3ZLf`).
4. Calls Razorpay API: `$api->order->create()` with `receipt=deposit.trx`, `amount=final_amount*100` (paise), `currency=INR`, `payment_capture=0`.
5. Stores Razorpay order ID in `deposit.btc_wallet`.
6. Returns JSON with checkout parameters (`key`, `amount`, `currency`, `order_id`, `prefill` user data, `theme` color).
7. Frontend renders Razorpay checkout.js modal.
8. User pays via UPI/card/netbanking in modal.
9. Razorpay redirects to `/ipn/razorpay?order_id={encrypted_btc_wallet}`.
10. `ProcessController::ipn()` decrypts `order_id`, loads deposit by `btc_wallet`, verifies HMAC SHA256 signature: `hash_hmac('sha256', order_id + '|' + razorpay_payment_id, key_secret) == razorpay_signature`.
11. If valid + `deposit.status=INITIATE`: calls `PaymentController::userDataUpdate()` which processes the order, credits balances, clears cart.
12. Redirects to `deposit.success_url (/checkout/thank-you)`.

PayPal Flow (Step-by-Step)

1. User selects PayPal → POST /checkout/payment/process with gateway=101. 2. CheckoutController creates Order + Deposit, calls PaypalProcessController::process(). 3. process() reads paypal_email from gateway_parameters. 4. Builds PayPal form fields: cmd=_xclick, business=paypal_email, currency_code, item_name, custom=deposit.trx, amount=final_amount (rounded 2 decimals), return=success_url, cancel_return=failed_url, notify_url=route('ipn.Paypal'). 5. Returns JSON with view='user.payment.redirect', url='https://www.paypal.com/cgi-bin/webscr', method='post'. 6. Frontend renders auto-submitting form to PayPal. 7. User pays on PayPal. 8. PayPal redirects back to /checkout/thank-you (success_url) — BUT this is NOT the confirmation. 9. PayPal sends IPN POST to /ipn/paypal asynchronously. 10. ProcessController::ipn() reads raw POST, parses key=value pairs, prepends cmd=_notify-validate, sends back to https://ipnpb.paypal.com/cgi-bin/webscr for VERIFIED check. 11. If VERIFIED: loads deposit by POST['custom'] (which is deposit.trx), checks mc_gross == deposit.final_amount (rounded 2 decimals), checks deposit.status=INITIATE, calls PaymentController::userDataUpdate(). 12. NOTE: PayPal IPN is DEPRECATED — PayPal recommends migrating to PayPal Webhooks API. Migration should use Stripe Checkout instead for international payments.

Manual UPI Flow (Step-by-Step)

1. User selects 'Google Pay UPI' in checkout. 2. Frontend shows QR code (tagadiyainfotech.jpg from gateway.description HTML). 3. User scans QR with Google Pay, sends payment, uploads screenshot. 4. Frontend POSTs to /deposit/manual with form data (screenshot URL, UPI transaction ID). 5. PaymentController::manualDepositUpdate() validates form via FormProcessor (dynamic validation against gateway.form.form_data JSON). 6. Stores form data in deposit.detail JSON. 7. Sets deposit.status=PENDING. 8. Creates AdminNotification 'Deposit request from {username}'. 9. Sends DEPOSIT_REQUEST email to user. 10. Admin visits /admin/payment/details/{id}, reviews screenshot, clicks Approve → admin/deposit/approve. 11. Admin's DepositController::approve() calls PaymentController::userDataUpdate(\$deposit, isManual=true) which processes the order same as automatic gateway.

Wallet Payment Flow

1. User has sufficient balance (e.g., from previous seller earnings). 2. User selects 'Wallet' as payment method (currently not exposed in UI but code path exists). 3. PaymentController::depositInsert() with gateway='wallet' validates balance >= amount. 4. Calls orderFromAccountBalance() which: sets order.payment_status=SUCCESS, deducts buyer.balance, creates buyer Transaction ('-' purchase), for each OrderItem: calculates seller_earning (NOTE: different formula from createOrder! BUG), credits author.balance, increments counters, creates author Transactions, syncs author levels. 5. Deletes cart. 6. Triggers referral commission. 7. Redirects to /checkout/thank-you.

Webhook Security Analysis

Razorpay: signature verified via HMAC SHA256 — secure IF key_secret is not compromised (currently EXPOSED in SQL dump — P0 rotation required). PayPal: no signature — relies on VERIFIED callback to PayPal's server. Vulnerable to MITM if PayPal URL is hijacked — but low risk in practice. Manual UPI: no webhook — admin manual approval. NestJS migration should: (1) verify Razorpay X-Razorpay-Signature header against webhook secret (different from API key secret), (2) migrate PayPal to Webhooks API with signature verification, (3) maintain manual UPI flow but add screenshot upload to S3 for audit trail.

Payment State Machine

Deposit status: 0=INITIATE (created, awaiting payment), 1=SUCCESS (paid), 2=PENDING (manual, awaiting admin approval), 3=CANCEL (failed/rejected). Order payment_status: 0=UNPAID, 1=PAID. Transitions: INITIATE → SUCCESS (gateway webhook confirms) OR INITIATE → PENDING (manual deposit submitted) → SUCCESS (admin approves) OR → CANCEL (admin rejects). OrderItem is_refunded: 0=NO, 1=YES. Refund flow creates RefundRequest, admin/seller approves, sets is_refunded=1, reverses transactions.

SECTION 7

Reviewer Workflow

Reviewer is a distinct role with subcategory-scoped permissions. 8 product queues, 4 rejection types, 2 update review types, atomic file swap on approval, activity log for audit trail. Critical workflow to replicate faithfully in NestJS.

Reviewer Role

Reviewers are a distinct role from admins and users. Stored in reviewers table with own password hash (bcrypt \$2y\$). Each reviewer has subcategories JSON field — array of sub_category IDs they're authorized to review. Reviewer can ONLY see products in their assigned subcategories. This is enforced in `productData()` private method: `Product::whereIn('sub_category_id', $reviewer->subcategories ?? []).` Currently only one reviewer exists (Pranav, assigned to subcategories [1,2,3,4,5,6,7] which covers all current subcategories).

Product Status State Machine

Products have a 'status' field with 6 states: 0=PENDING (newly uploaded, awaiting review), 1=APPROVED (live on site), 2=SOFT_REJECTED (needs fixes, seller can edit and resubmit), 3=HARD_REJECTED (permanently rejected, no resubmission), 4=DOWN (temporarily off-site, seller can fix), 5=PERMANENT_DOWN (permanently removed but file kept for existing buyers). Additionally, 'product_updated' field tracks update state: 0=NO_UPDATE, 1=PENDING (seller uploaded new file, awaiting review), 2=APPROVED, 3=SOFT_REJECT, 4=HARD_REJECT. The two fields are independent — an APPROVED product can have a PENDING update.

Eight Reviewer Queues

Reviewer dashboard has 8 distinct queues: (1) `pendingProducts` — products in their subcategories, `status=PENDING`, `assigned_to=0` OR `assigned_to=reviewer`. (2) `awaitingProducts` — `status=PENDING` OR `product_updated=PENDING` (excluding `PERMANENT_DOWN`, `HARD_REJECTED`, `UPDATE_HARD_REJECT`). (3) `assignedProducts` — `assigned_to=current reviewer`. (4) `approvedProducts` — `status=APPROVED`, reviewed by current reviewer. (5) `softRejectedProducts` — `status=SOFT_REJECTED` by current reviewer. (6) `hardRejectedProducts` — `status=HARD_REJECTED` by current reviewer. (7) `downProducts` — `status=DOWN` by current reviewer. (8) `permanentDownProducts` — `status=PERMANENT_DOWN` by current reviewer. Plus `updatedProducts` — products with `product_updated=ENABLE` awaiting update review.

Assignment Flow

Reviewer visits `pendingProducts` queue. Clicks 'Assign to me' on a product → POST `/reviewer/product/{slug}/assign`. `assignProduct()` validates: product must be unassigned (`assigned_to=0`) AND `status=PENDING`. Sets `assigned_to=auth_id`. Other reviewers can no longer see this product in their pending queue. Reviewer now sees it in their `assignedProducts` queue. Reviewer clicks product → details page → can download current file + temp file (if update pending) for review.

Approval Flow

Reviewer clicks 'Approve' → POST /reviewer/product/{id}/approve. approveProduct() does ATOMIC FILE SWAP: old product.file is replaced by product.temp_file (the newly uploaded version). temp_file set to null. Sets approved_by=reviewer_id, published_at=now, status=APPROVED. Creates Activity with '[Approve]Your product has been approved' message. Sends PRODUCT_APPROVED email to author IF author.email_settings.review_notification is enabled.

Rejection Flow (4 Types)

Reviewer can reject with 4 types: (1) SOFT_REJECTED (type=2) — reason required, seller can edit and resubmit. (2) HARD_REJECTED (type=3) — reason optional, permanently rejected, file physically deleted IF no sales exist. (3) DOWN (type=4) — reason required, product temporarily off-site. (4) PERMANENT_DOWN (type=5) — message required, permanently removed but file kept for existing buyers. Each rejection creates Activity with appropriate prefix ('[Soft Reject]', '[Hard Reject]', '[Down]', '[Permanent Down]'). Sends corresponding email notification.

Update Approval/Rejection

When seller uploads new main_file for existing product: product_updated set to PENDING, new file stored in temp_file (NOT file — old file remains active for buyers). Reviewer sees product in updatedProducts queue. Approve update: POST /reviewer/product/update/{id}/approve → approveUpdate() does FILE SWAP: deletes old product.file, moves temp_file → file, sets product_updated=NO_UPDATE, status=APPROVED, last_updated=now. Reject update: POST /reviewer/product/update/{id}/reject/{type} (type=3 soft or 4 hard) → rejectUpdate() deletes temp_file, sets product_updated=type, keeps old file active.

File Lifecycle

Seller uploads → temp_file (NOT file). On approval: temp_file → file (atomic swap). On hard reject / permanent down with no sales: file physically deleted via FileUploader. On permanent down WITH sales: file kept (existing buyers can still download). On seller re-upload after approval: new file → temp_file, product_updated=PENDING. Old file remains active until reviewer approves update. NestJS migration should use S3 with versioning — old versions recoverable if needed.

Activity Log

Every reviewer action creates an Activity row: message (prefixed with action type in brackets), product_id, reviewer_id, status (matching new product status), timestamps. Seller can reply via /product/{slug}/activities → creates Activity with user_id (no reviewer_id). Activity log visible to both seller and reviewer on product details page. Used for audit trail and dispute resolution.

SECTION 8

Admin Panel Flow

Admin panel has 30+ modules with 200+ routes (508 lines of admin routes). Single admin role (no granular permissions) with impersonation capability. This section provides module-level inventory — full route documentation is in `routes/admin.php`.

8.1 Admin Module Inventory

Module	Routes Count	Key Controllers	Notes
Dashboard & Reports	8	AdminController	Dashboard widgets, sales chart, deposit/withdraw chart, transaction chart, top selling items
Blog Management	6	BlogCategoryController, BlogPostController	Categories (CRUD), Posts (CRUD)
Category Management	10	CategoryController, SubCategoryController	Categories + subcategories, SEO per category, reviewer assignment per subcategory
Review Categories	5	ReviewCategoryController	Categories for product reviews (e.g., 'Code Quality', 'Documentation')
Author Management	5	ManageAuthorController, AuthorLevelController	Author list, status toggle, feature toggle, author levels (gamification tiers)
Comment Moderation	6	CommentController	List comments, view replies, delete, show/hide
Review Moderation	5	ReviewController	List reviews, delete, show/hide, download attachments
Product Management	9	ManageProductController	View by status (pending/approved/soft-rejected/hard-rejected/down/permanent-down/all), details, toggle feature
Reviewer Management	6	ManageReviewerController	CRUD reviewers, login as reviewer (impersonation), sync subcategories
Advertisement Management	4	ManageAdsController	Create/edit ads, toggle status
Coupon Management	5	ManageCouponsController	Create/edit coupons, toggle status
Sales Reports	6	SellController	All sales, sale items by author, refunded items, details
Campaign Management	8	CampaignController	Create/edit campaigns, approve/reject submitted products
User Management	20+	ManageUsersController	List by segment (active/banned/verified/unverified/with-balance), detail, KYC approve/reject, add/sub balance, send notification, login as user (impersonation), status toggle
Subscriber Management	4	SubscriberController	List, send email, remove
Payment Gateways	12	AutomaticGatewayController, ManualGatewayController	Automatic gateways (Razorpay/PayPal/etc.) + manual gateways (UPI/etc.), edit config, toggle status
Deposit Management	9	DepositController	List by status (pending/approved/rejected/successful/initiated), details, approve, reject

Module	Routes Count	Key Controllers	Notes
Withdrawal Management	10	WithdrawalController, WithdrawMethodController	List withdrawals by status, approve, reject. Manage withdraw methods (CRUD)
Reports	5	ReportController	Transaction report, login history, IP history, notification history, email details
Support Tickets	8	SupportTicketController	List by status (pending/closed/answered), view, reply, close, delete, download attachment
Language Management	10	LanguageController	Manage languages, import/export, edit keys, delete keys
File Storage	8	FileStorageController	Configure FTP, Wasabi, DigitalOcean Spaces, Backblaze credentials
General Settings	20+	GeneralSettingController	Site name, currency, email config, SMS config, socialite credentials, system config, logo/icon, custom CSS, sitemap, robots, cookie, maintenance mode
Author Info Form	2	AuthorInfoController	Configure dynamic form for author application
KYC Settings	2	KycController	Configure KYC form fields
Cron Configuration	9	CronConfigurationController	Manage cron jobs (CRUD), schedules, view logs, resolve errors, flush logs
Notification Templates	14	NotificationController	Global email/SMS/push settings, template editing, email/SMS test, push config (Firebase)
Extensions (Plugins)	3	ExtensionController	List, update, toggle status for installed extensions
System Info	7	SystemController	System info, server info, optimize cache, system update, update log
Frontend Page Builder	12	FrontendController, PageBuilderController	Manage templates, frontend sections, page builder, SEO per page

SECTION 9

Notification System

Multi-channel notification system (email + SMS + push) via centralized notify() helper. 20+ templates with shortcode substitution. Author email settings control per-notification opt-in. NestJS migration should use a NotificationService with pluggable channel providers.

Multi-Channel Architecture

The notify() helper function is the single entry point for all notifications. Signature: notify(\$user, \$templateName, \$shortCodes, \$sendVia, \$createLog, \$pushImage). It instantiates a Notify object with the channels to send via (email, sms, push — passed as array OR pulled from template config). The Notify class loads the NotificationTemplate by act=\$templateName, renders the email_body/sms_body/push_body with shortcode substitution, and sends via configured gateways. Every notification is logged to notification_logs table with user_id, sender, sent_from, sent_to, subject, message, notification_type, image, user_read flag.

Template System

notification_templates table stores templates. Each template has: act (unique key, e.g., 'DEPOSIT_COMPLETE', 'PRODUCT_APPROVED', 'PRODUCT_SOFT_REJECTED'), name, subject, push_title, email_body (HTML), sms_body (plain text), push_body (plain text), shortcodes (JSON list of supported placeholders), email_status/sms_status/push_status (toggle per channel), email_sent_from_name, email_sent_from_address, sms_sent_from. Templates use {{shortcode}} syntax. Global shortcodes always available: site_name, site_currency, currency_symbol.

Email Channel

Email rendered from template.email_body with shortcode substitution. Sent via PHP mailer configured in general_settings.mail_config (JSON). Currently set to {"name":"php"} — uses PHP mail() function. Schema supports SMTP, SendGrid, Mailjet, etc. NestJS migration should use AWS SES via nodemailer. Global email wrapper template in general_settings.email_template wraps every email with site branding (logo, footer). Shortcodes in wrapper: {{fullname}}, {{username}}, {{message}}, {{site_name}}.

SMS Channel

SMS rendered from template.sms_body. Sent via configured SMS gateway in general_settings.sms_config. Schema supports 6 gateways: Nexmo, Twilio, MessageBird, Clickatell, TextMagic, SMS Broadcast, plus a Custom HTTP API gateway. Currently disabled (sn=0 in settings). NestJS migration should use Twilio SDK directly.

Push Channel

Push rendered from `template.push_title + push_body`. Sent via Firebase Cloud Messaging (FCM). Firebase config in `general_settings.firebase_config` (currently all placeholders). User device tokens stored in `device_tokens` table. User registers device via `POST /user/add-device-token`. Currently disabled (`pn=1` in settings but `firebase_config` not configured). NestJS migration should use `firebase-admin` SDK.

Notification Triggers

Notifications triggered at key moments: registration (welcome email), email/mobile verification (code sent), password reset (code sent), 2FA enable/disable, KYC submit/approve/reject, product approval/rejection (5 templates: `PRODUCT_APPROVED`, `PRODUCT_SOFT_REJECTED`, `PRODUCT_HARD_REJECTED`, `PRODUCT_DOWN`, `PRODUCT_PERMANENT_DOWN`), product update approved/rejected (2 templates), deposit complete/request/approve (3 templates), withdrawal request/approve/reject (3 templates), refund request (1 template), support ticket reply (1 template), referral commission (1 template). Total: ~20 notification templates.

Author Email Settings

Each author (seller) has `email_settings` JSON field on user record. Controls which notifications they receive: `review_notification` (product approval/rejection), `sale_notification` (new sale), `refund_notification` (refund request), `comment_notification` (new comment on product), `review_notification` (new review on product). User toggles these in profile settings. `notify()` checks these flags BEFORE sending.

SECTION 10

Cron Jobs & Scheduled Tasks

DB-driven cron system with admin UI. Master /cron endpoint hit every minute by system crontab. Two job types: internal controller methods (is_default=1) and external URLs (is_default=0). Currently only campaignExpired is explicitly defined in source code.

Cron Architecture

Cron system is database-driven. cron_schedules table defines intervals (in seconds). cron_jobs table stores individual jobs with: name, alias, action (JSON array [ControllerClass, methodName] for default jobs OR url for external jobs), cron_schedule_id, next_run, last_run, is_running, is_default. The master /cron endpoint is hit by system crontab every minute. CronController::cron() loads all due jobs, executes each, logs to cron_job_logs with start_at, end_at, duration, error.

Job Types

Two types: (1) is_default=1 — internal controller method. CronController instantiates the controller and calls the method. Example: [CronController::class, 'campaignExpired']. (2) is_default=0 — external URL. CronController calls CurlRequest::curlContent(\$cron->url) to hit the URL. Used for tasks like external API syncs.

Known Active Crons

Based on cron_jobs table in SQL dump + CronController: (1) campaignExpired — runs daily, finds campaigns past end_date, marks as expired. (2) general_settings.last_cron — updated every cron run (used for health monitoring). Additional cron jobs may exist in cron_jobs table but source code only shows campaignExpired explicitly. Full cron inventory requires querying cron_jobs table in production (currently blocked — only have SQL dump).

Manual Cron Triggering

Admin can manually trigger cron jobs via /admin/cron/index. CronController supports ?alias=X for specific job, ?target=all for all jobs. Admin UI shows next_run, last_run, is_running status, logs with error messages, allows pausing schedules, resolving log errors, flushing logs.

NestJS Migration Target

Two options: (1) AWS EventBridge + Lambda for serverless cron — best for cost, scales to zero. (2) BullMQ repeatable jobs on NestJS worker process — best for tasks that need DB access. Recommendation: use EventBridge for time-based triggers (campaign expiry, daily reports), BullMQ for task-based triggers (email sending, image processing). Both should log to a cron_job_logs equivalent table for audit trail.

SECTION 11

Helpers & Utilities

766-line helpers.php file contains 35+ utility functions used throughout the application. These must be replicated as NestJS services/utilities. Key helpers: notify(), fileUploader(), getTrx(), getRealIP(), showAmount(), showDateTime(), verifyG2fa().

11.1 Helper Function Inventory

Helper Function	Purpose	NestJS Equivalent
systemDetails()	Returns codeplus v2.3 build 5.1.9. Used for license verification.	Config service — read from package.json or env
slug(\$string)	Wrapper for Str::slug(). URL-friendly slug.	slugify npm package
verificationCode(\$length)	Generates N-digit numeric code (e.g., 6-digit for 2FA).	Use crypto.randomInt(min, max)
getNumber(\$length=8)	Generates random numeric string of given length.	Use crypto.randomBytes + filter digits
activeTemplate(\$asset=false)	Returns active template name from session or general_settings. Used for Blade namespace.	Not needed — Next.js uses component-based templates
siteLogo(\$type=null)	Returns logo URL. Supports type variants (e.g., 'footer', 'dark').	Config service — read from site settings
siteFavicon()	Returns favicon URL.	Config service
loadReCaptcha()	Returns Google reCaptcha script HTML.	@react-google-recaptcha npm package
loadCustomCaptcha(\$w, \$h, \$bg)	Returns custom captcha HTML (math/text-based).	Use svg-captcha npm package
verifyCaptcha()	Verifies captcha response. Returns bool.	Server-side: verify with Google API or custom check
loadExtension(\$key)	Loads extension script by key (e.g., 'recaptcha', 'analytics').	Plugin system — read from extensions table
getTrx(\$length=12)	Generates uppercase alphanumeric transaction ID.	Use nanoid or short-uuid npm
getAmount(\$amount, \$length=2)	Rounds to N decimals, returns as number (not string).	Decimal.js round()
showAmount(\$amount, \$decimal, \$separate, \$exceptZeros, \$currencyFormat)	Formats amount with currency symbol/text based on general_settings.currency_format.	Intl.NumberFormat with currency option
cryptoQR(\$wallet)	Returns QR code URL for crypto wallet via qrserver.com API.	qrcode npm package for self-hosted QR
keyToTitle(\$text)	Converts 'snake_case' to 'Title Case'.	lodash.startCase()
titleToKey(\$text)	Converts 'Title Case' to 'snake_case'.	lodash.snakeCase()
strLimit(\$title, \$length)	Wraps Str::limit(). Truncates with ellipsis.	lodash.truncate()
getIpInfo()	Returns IP geolocation info via ClientInfo library.	Use ipapi.co or maxmind GeoIP2
osBrowser()	Returns browser + OS from User-Agent via ClientInfo library.	Use useragent npm package
getImage(\$image, \$size)	Returns image URL if exists, else placeholder image URL.	Image service — check S3 existence, return CloudFront URL or placeholder

Helper Function	Purpose	NestJS Equivalent
notify(\$user, \$templateName, \$shortCodes, \$sendVia, \$createLog, \$pushImage)	THE BIG ONE. Sends multi-channel notification from template.	NotificationService.send(user, template, shortcodes, channels)
getPaginate(\$paginate=null)	Returns pagination size from general_settings.paginate_number.	Config service — read from site settings
fileUploader(\$file, \$location, \$size, \$old, \$thumb, \$filename)	Uploads file via FileManager. Handles thumbnails, old file deletion.	S3Service.upload() with image processing via sharp
fileManager()	Returns new FileManager instance. Magic method — exposes file paths/sizes by key.	FileConfigService — read from config table
getFilePath(\$key)	Returns file path for key (e.g., 'productFile', 'productThumbnail').	FileConfigService.getPath(key)
getFileSize(\$key)	Returns max file size for key.	FileConfigService.getSize(key)
diffForHumans(\$date)	Returns '3 days ago' style time difference. Localized.	date-fns formatDistanceToNow
showDateTime(\$date, \$format)	Formats datetime. Localized via Carbon.	date-fns format with locale
verifyG2fa(\$user, \$code, \$secret)	Verifies Google Authenticator TOTP code.	otplib authenticator.verify
urlPath(\$routeName, \$routeParam)	Returns URL path (without domain) for named route.	Not needed — Next.js uses Link component
getRealIP()	Returns client IP. Checks HTTP_FORWARDED, HTTP_X_FORWARDED_FOR, HTTP_CLIENT_IP, HTTP_CF_CONNECTING_IP, etc.	req.ip or custom middleware for proxy headers
getContent(\$dataKeys, \$singleQuery, \$limit, \$orderById)	Loads Frontend (UI customization) content by data_keys.	FrontendService.getContent(keys)
getDefaultLang()	Returns default language code from Language model.	Config service — read from languages table
menuActive(\$routeName, \$type, \$param)	Returns CSS class for active menu item.	Not needed — Next.js usePathname() hook

SECTION 12

Updated Schema Decision Record

Refined schema decisions based on source code analysis. Where Part A and Part B differ, Part B is authoritative. Key changes: carts kept as-is (no parent entity), general_settings split into 7 typed tables, activities get action_type enum, deposits get webhook_event_id for idempotency.

12.1 Refined Schema Decisions

Table	Part A Decision	Part B Refined Decision	Reason
users	KEEP + ADD COLUMN (auth_provider, auth_provider_id, password_algo, last_login_at)	KEEP + ADD: auth_provider, auth_provider_id, password_algo (default 'bcrypt-2y'), last_login_at, email_settings JSON, social_media_settings JSON, is_author_featured, tsc (2FA secret)	Source code revealed email_settings + social_media_settings + is_author_featured + tsc fields exist and are actively used. All must be preserved.
carts	REFACTOR (introduce Cart parent entity)	KEEP AS-IS (no parent Cart entity)	Source code confirmed cart is intentionally per-item. Refactoring would change business logic. Migration must preserve current model.
orders	KEEP + ADD COLUMN (gateway, gateway_transaction_id, currency_code, ip_address, user_agent)	KEEP + ADD: gateway_method_code, gateway_trx, currency_code, ip_address, user_agent, checkout_session_id	Source code showed deposit.btc_wallet reused for Razorpay order_id — confusing. New schema should separate concerns.
deposits	RENAME to Payment	RENAME to Payment + ADD: webhook_event_id (for idempotency), gateway_response JSON, verified_at, verified_by	Source code confirmed is_web field was for abandoned Next.js attempt. Webhook idempotency is critical for NestJS.
products	KEEP + ADD COLUMN (search_index_updated_at, prisma_search_vector)	KEEP + ADD: search_index_updated_at, meilisearch_doc_id, view_count_cached (from product_views aggregation)	Source code showed ProductView aggregation done on-demand in isTrending() — should be cached for performance.
reviews	KEEP	KEEP + ADD: is_synthetic (flag for auto-generated reviews), country_hint	Source code confirmed reviews:add-foreign Artisan command generates fake reviews. Migration should preserve but flag them.
activities	KEEP	KEEP + ADD: action_type (enum: APPROVE, SOFT_REJECT, HARD_REJECT, DOWN, PERMANENT_DOWN, UPDATE_APPROVE, UPDATE_SOFT_REJECT, UPDATE_HARD_REJECT, REPLY)	Source code parses message prefix '[Approve]' '[Soft Reject]' etc. to determine action. New schema should use enum instead.
gateways	KEEP	KEEP + ADD: webhook_secret (separate from API key), supported_features JSON	Source code showed Razorpay uses key_secret for both API auth AND webhook signature. Should be separated for security.
notification_logs	RENAME to NotificationLog	KEEP + ADD: correlation_id (for grouping multi-channel sends), retry_count, error_message	Source code showed notify() can fail silently on one channel. New schema should track per-channel delivery status.

Table	Part A Decision	Part B Refined Decision	Reason
general_settings	KEEP (singleton)	SPLIT into multiple typed tables: SiteConfig, EmailConfig, SmsConfig, PushConfig, StorageConfig, SeoConfig, SocialiteConfig	Source code showed 50+ fields crammed into one row. Splitting improves type safety and reduces Prisma model complexity.
product_views	EVALUATE (maybe Redis)	KEEP (not Redis) — used for trending calculation in isTrending()	Source code confirmed daily aggregation query. Redis hyperloglog would lose daily granularity needed for trending.
forms	KEEP (DynamicForm)	KEEP + ADD: form_submissions table (currently form data scattered across deposits.detail, users.kyc_data, users.author_info)	Source code showed form submissions stored in different JSON fields per use case. Should centralize.
user_follows	EVALUATE (duplicate of followers?)	DEPRECATE — followers table is canonical (confirmed in User model relations)	Source code confirmed both tables exist but User model only uses followers. user_follows is legacy/dead.
cron_jobs	KEEP (or migrate to EventBridge)	MIGRATE to BullMQ repeatable jobs + EventBridge. Keep cron_jobs table for backward compat during transition.	Source code showed cron system is DB-driven with admin UI. Migration should preserve admin UX but use modern scheduler.

SECTION 13

Updated NestJS Module Map

Refined NestJS module map with source controllers mapped to target services. 20 modules total (2 more than Part A — CronModule and SupportModule split out for clarity). Each module lists the Laravel controllers it absorbs and the key services to extract.

13.1 Module → Controller → Service Mapping

NestJS Module	Source Controllers Mapped	Key Services to Extract
AuthModule	LoginController, RegisterController, ForgotPasswordController, ResetPasswordController, SocialiteController, AuthorizationController, Admin/Auth/*, Reviewer/Auth/*	AuthService, PasswordService (bcrypt rehash), OAuthService, TwoFactorService, EmailVerificationService, MobileVerificationService, ImpersonationService
UserModule	UserController, ProfileController, UserController@userData, UserController@kyc*, UserController@twofactor*, UserController@addDeviceToken	UserService, ProfileService, KycService, DeviceTokenService, UserLoginService (IP/geo logging)
ProductModule	User\ProductController, Reviewer\ProductController, Admin\ManageProductController	ProductService, ProductUploadService (S3), ProductApprovalService, ProductStatusService, ChangelogService, ScreenshotExtractorService
CategoryModule	Admin\CategoryController, Admin\SubCategoryController	CategoryService, SubCategoryService, CategorySeoService
CartModule	CartController	CartService, CartPricingService (license + addon + campaign), CartTransferService (session→user), CouponService
CheckoutModule	CheckoutController	CheckoutService, OrderCreationService (atomic), OrderPricingService, CheckoutReturnService
OrderModule	User\OrderController, Admin\SellController	OrderService, OrderItemService, PurchaseCodeService (generation + verification), DownloadService
PaymentModule	Gateway\PaymentController, Gateway\Razorpay\ProcessController, Gateway\Paypal\ProcessController, Admin\DepositController, Admin\AutomaticGatewayController, Admin\ManualGatewayController	PaymentService, RazorpayService, PaypalService, ManualUpiService, WalletPaymentService, PaymentWebhookService (idempotent), GatewayConfigService
ReviewModule	User\AuthorController@saveReview, User\AuthorController@saveComment, User\AuthorController@reportReview, User\AuthorController@reportComment, Admin\ReviewController, Admin\CommentController	ReviewService, CommentService (nested), ReviewModerationService, ReviewAggregationService (updates product.avg_rating)
BlogModule	BlogController, Admin\BlogCategoryController, Admin\BlogPostController	BlogService, BlogSeoService
PageModule	SiteController@pages, SiteController@policyPages, Admin\FrontendController, Admin\PageBuilderController	PageService, FrontendService (UI customization), PolicyPageService
AuthorModule	User\AuthorController, Admin\ManageAuthorController, Admin\AuthorLevelController, Admin\AuthorInfoController	AuthorService, AuthorLevelService (gamification), AuthorEarningsService, FollowService, CollectionService
ActivityModule	User\ProductController@productActivities, User\ProductController@replyActivity, Reviewer\ProductController@replyActivity	ActivityService, ProductApprovalService (orchestrates file swap + activity + notification)

NestJS Module	Source Controllers Mapped	Key Services to Extract
AdminModule	Admin\AdminController + 20+ other admin controllers	AdminAuthService, AdminDashboardService, AdminUserService (with impersonation), AdminProductService, AdminDepositService, AdminWithdrawalService, AdminReportService, AdminSettingsService
SettingsModule	Admin\GeneralSettingController, Admin\SystemController, Admin\FileStorageController, Admin\LanguageController, Admin\ExtensionController	SiteConfigService, EmailConfigService, SmsConfigService, StorageConfigService, LanguageService, ExtensionService, SystemInfoService
NotificationModule	(no controller — helper-based via notify())	NotificationService, EmailService (SES), SmsService (Twilio), PushService (FCM), TemplateRendererService, NotificationLogService
MediaModule	(no controller — helper-based via fileUploader())	S3Service, CloudFrontService, ImageProcessingService (sharp), FileUploadService, SignedUrlService (1-hour expiry for APK), DownloadLogService
SupportModule	TicketController, Admin\SupportTicketController	SupportTicketService, SupportMessageService, SupportAttachmentService (S3 upload)
CronModule	CronController, Admin\CronConfigurationController	CronService (BullMQ), CronLogService, CampaignExpiryService, ScheduledTaskService
SharedModule	(global)	PrismaService, RedisService, QueueService (BullMQ), LoggerService (structured JSON + correlation ID), ConfigService, HashService, PaginationService, ResponseService, ErrorFilter, ValidationPipe, LoggingInterceptor, RateLimiterGuard

SECTION 14

20 Open Questions — Answered

All 20 open questions from Part A Section 11.2 are now answered using the Laravel source code. These answers inform the NestJS implementation and supersede any assumptions in Part A.

1. Exact Laravel middleware stack

14 middleware confirmed: MaintenanceMode, LanguageMiddleware, ActiveTemplateMiddleware (global); Authenticate, RedirectIfAuthenticated, CheckStatus, RegistrationStep, KycMiddleware, AuthorMiddleware, RedirectIfAdmin, RedirectIfNotAdmin, RedirectIfReviewer, RedirectIfNotReviewer, Demo (route-specific); VerifyCsrfToken (excludes /ipn/*).

2. Validation rules per controller

Extracted — see Section 2. Most validation is inline in controller methods (not FormRequest classes). Key finding: contact form has NO email format validation (potential bug). Product save has complex dynamic validation via FormProcessor for subcategory attributes.

3. Blade templates for admin panel

Confirmed — admin views in resources/views/admin/. Layout: admin/layouts/master.blade.php. Pages: dashboard, profile, password, notifications, reports, top_selling_products. Plus 30+ admin subdirectories for each module (category, product, user, deposit, withdrawal, etc.).

4. Email template rendering logic

notify() helper → Notify class → loads NotificationTemplate by act → renders email_body with shortcode substitution ({{shortcode}} → value). Global wrapper in general_settings.email_template wraps every email. Shortcodes: site_name, site_currency, currency_symbol (global) + per-template shortcodes.

5. Queued jobs

NO QUEUED JOBS. Application does NOT use Laravel Queue. Everything is synchronous. Email sending, image processing, file uploads — all happen in request lifecycle. NestJS migration should move these to BullMQ for performance.

6. Scheduled tasks (cron)

Confirmed 1 explicit: campaignExpired (CronController method). Plus DB-driven cron system supports admin-configured jobs. /cron endpoint hit by system crontab. Other cron jobs may exist in cron_jobs table but source code only shows campaignExpired.

7. Laravel Scout / search config

NO LARAVEL SCOUT. Search uses MySQL LIKE + JSON contains (for tags). Custom searchable() scope in some controllers. NestJS migration should use Meilisearch as planned.

8. Socialite provider details

Google: client_id 174714211-v1uh0r6tp3fn6kb9j0jucvk5tevc365s (LIVE — must rotate), status=1 (active). Facebook: status=0 (disabled). LinkedIn: status=0 (disabled). SocialLogin library handles the flow.

9. File upload rules

Per category: file_type (script/audio/video), file_size (MB), preview_file_types (array of allowed extensions). main_file: ZIP only. screenshots: ZIP containing PNG/JPG/JPEG only (no subdirectories). thumbnail/preview_image: image files. preview_video: URL (not file upload). Max sizes from category.file_size + general_settings.preview_video_size.

10. Coupon calculation logic

Validates: exists, status=1, within date range, subtotal >= minimum_spend, total usage < usage_limit_per_coupon, user usage < usage_limit_per_user. Discount: percent (type=2) → (price + buyer_fee + extended_amount) * amount / 100. Fixed (type=1) → amount. Applied per cart item, stored in cart.discount. Coupon cleared on any cart modification.

11. Seller fee calculation

Per OrderItem: author's highest AuthorLevel where minimum_earning <= author.total_sold_amount. If no level: lowest minimum_earning level. seller_fee = level.fee (percentage) * cartItem.price. Deducted from author balance in separate Transaction (type='-', remark='seller_fee').

12. Refund approval workflow

Buyer initiates via /author/refund/request/{purchase_code} → creates refund_request with reason. Seller notified. Seller can accept (credits buyer, deducts seller, sets is_refunded=1) or reject (creates refund_activity log). All messages logged in refund_activities table. Admin can also intervene.

13. Reviewer subcategory assignment logic

Stored as JSON array in reviewers.subcategories. Enforced in Reviewer\ProductController::productData() via whereIn('sub_category_id', \$reviewer->subcategories ?? []). Admin can sync via /admin/reviewers/subcategories/{id} → ManageReviewerController::syncSubcategories().

14. Author level promotion logic

After every sale (in PaymentController::userDataUpdate and orderFromAccountBalance): queries AuthorLevel::active()->where('minimum_earning', '<=', author.total_sold_amount)->pluck('id'). Syncs to author_level_user pivot via \$author->authorLevels()->sync(\$authorLevels). Highest applicable level determines fee.

15. Notification template rendering

notify() loads template by act field. Renders email_body/sms_body/push_body with shortcode substitution. Checks author.email_settings flags BEFORE sending. Logs to notification_logs. Channels sent in parallel (or per template config).

16. Existing API routes

Only 4 endpoints: POST /api/verify-purchase-code (public), GET /api/author/products/all (API key), GET /api/author/product/details (API key). is_web flag in deposits was for abandoned mobile app or Next.js attempt — no production API to reuse.

17. Admin role permissions

Single admin role — no granular permissions. All admins can do everything. Admin can impersonate users (/admin/users/login/{id}) and reviewers (/admin/reviewers/login/{id}). NestJS migration should add RBAC with roles like super-admin, content-admin, finance-admin, support-admin.

18. Blog post body rendering

Longtext stored in blog_posts.body. Rendered as-is (HTML) in Blade template {!! \$post->body !!}. NO HTML sanitization for blog posts (potential XSS). NestJS migration should sanitize with DOMPurify on render OR store as Markdown and render with react-markdown.

19. Product slug generation

Via productSlug() helper (custom, not Str::slug directly). Called in User\ProductController::saveProduct when creating new product. Slug stored in products.slug. Unique constraint enforced at DB level. URL: /game-source-code/{slug}.

20. Cart session ID generation

Laravel session ID via session()->getId(). Stored in carts.session_id for guest carts. On login/register: transferCartItemsToUser() reads pre_login_session_id from session, queries Cart::where('session_id', pre_login_session_id), updates rows to set user_id + null session_id. NestJS migration: use Redis key like cart:session:{uuid} for guests, cart:user:{userId} for auth.

SECTION 15

Updated Risk Register (New Risks)

14 new risks discovered during source code analysis (R13-R26). These are added to the 12 risks from Part A (R1-R12) for a total of 26 risks. The most critical new risks: R13 (seller earning formula inconsistency), R14 (no queued jobs), R20 (PayPal IPN deprecated).

15.1 New Risks (R13-R26)

#	New Risk	Likelihood	Impact	Mitigation
R13	Seller earning formula inconsistency between createOrder() and userDataUpdate()	HIGH	HIGH	Source code has TWO different formulas for seller_earning. Migration MUST pick ONE and document. Recommended: (price - (seller_fee + discount)) + extended_amount + addon_services_amount. Audit existing order_items for discrepancies before migration.
R14	No queued jobs — all email/file/notifications synchronous	HIGH	MEDIUM	Source code confirmed zero queue usage. Request lifecycle includes email sending + file uploads. NestJS migration MUST move to BullMQ — otherwise p95 latency will be 5-10s.
R15	Blog post body has NO XSS sanitization	MEDIUM	HIGH	Source code renders blog body with {!! \$post->body !!} (raw HTML). Stored XSS possible if admin enters malicious script. NestJS migration should sanitize with DOMPurify OR convert to Markdown.
R16	Contact form has no email format validation	LOW	LOW	Source code: 'email' => 'required' (no email format rule). Spam tickets with invalid emails can be created. NestJS migration should add Zod email validation.
R17	Reviews in production DB are partially synthetic	HIGH	MEDIUM	reviews:add-foreign Artisan command generated fake reviews with country hints. Migration must preserve (user-facing) but new schema should flag with is_synthetic boolean. Disable command post-cutover.
R18	Sales counts (total_sold) are partially synthetic	HIGH	MEDIUM	products:set-random-sales command sets fake counts. Migration must preserve but new schema should add is_sales_count_verified flag. Disable command post-cutover.
R19	Admin can impersonate users/reviewers without audit trail	MEDIUM	MEDIUM	Source code: Auth::login(\$targetUser) — replaces admin session entirely. No log of who impersonated whom. NestJS migration should use proper impersonation token with impersonated_by claim + audit log.
R20	PayPal IPN is deprecated	HIGH	HIGH	Source code uses PayPal IPN (cmd=_notify-validate). PayPal has deprecated IPN in favor of Webhooks API. NestJS migration MUST use PayPal Webhooks OR replace with Stripe Checkout (recommended).
R21	Razorpay key_secret used for both API auth AND webhook signature	MEDIUM	MEDIUM	Source code: same key_secret used for API calls and HMAC webhook verification. If key compromised, attacker can forge webhooks. NestJS migration should use separate webhook_secret (Razorpay supports this).
R22	Manual UPI flow has no automated verification	MEDIUM	MEDIUM	Source code: admin manually verifies screenshot + approves. Slow, error-prone. NestJS migration should explore Razorpay UPI deep links (auto-verified) OR keep manual but add S3 screenshot upload for audit.
R23	No rate limiting on auth endpoints	MEDIUM	HIGH	Source code uses Laravel's ThrottlesLogins trait (5 attempts/min) for login — but NO rate limit on registration, password reset, contact form. NestJS migration should add @nestjs/throttler on ALL auth endpoints.

#	New Risk	Likelihood	Impact	Mitigation
R2 4	Product file lifecycle is fragile (temp_file → file swap)	MEDIUM	MEDIUM	Source code does atomic file rename on approval. If process crashes mid-swap, file state inconsistent. NestJS migration should use S3 versioning + database transaction for atomicity.
R2 5	Multi-language infrastructure exists but is unused	LOW	LOW	Source code has LanguageMiddleware, Language model, admin language management. Currently only English. NestJS migration should preserve i18n infrastructure (next-intl) but not implement translations.
R2 6	Referral system code exists but is disabled	LOW	LOW	Source code: Referral::processReferralCommission() invoked on every sale IF enabled. Currently disabled (referral=0). NestJS migration should preserve code path but keep disabled.

SECTION 16

Migration Decision Records (20 MDRs)

Formal decision records for every architectural choice that differs from Part A or that requires explicit approval. Each MDR has a number, decision, rationale, and approver. MDRs are immutable once approved — changes require a new MDR that supersedes the prior one.

16.1 Migration Decision Records

MDR #	Decision	Rationale	Approved By
MDR-001	Use password rehash on first login (not clean break, not dual-token bridge)	Zero user disruption. bcrypt \$2y\$ verified by Node bcrypt. Rehash to \$2b\$ on success. No session invalidation. Approved in Phase 0 Q2.	User (Phase 0 Q2)
MDR-002	Use Prisma ORM (not TypeORM)	Cleaner schema DSL, better type inference, mature migrations. Approved in Phase 0 Q3.	User (Phase 0 Q3)
MDR-003	Deploy on AWS (ECS Fargate + RDS + ElastiCache + S3 + CloudFront)	Scales to 10k+ concurrent users. Multi-AZ HA. Approved in Phase 0 Q4.	User (Phase 0 Q4)
MDR-004	Keep Razorpay + PayPal during migration, add Stripe Checkout post-cutover	Preserves existing payment flow. PayPal IPN deprecated — Stripe Checkout is modern replacement. Approved in Phase 0 Q5.	User (Phase 0 Q5)
MDR-005	Strangler-fig migration (not big-bang)	Per-route rollback. SEO preservation. 30-week timeline. Approved in Phase 0 Q6.	User (Phase 0 Q6)
MDR-006	Drop speculative modules (Wallet, Leaderboard, Transaction)	Marketplace has no wallets or leaderboards. Transactions fold into Order module. Approved in Phase 0 Q1.	User (Phase 0 Q1)
MDR-007	Use Meilisearch for search (not MySQL LIKE, not Elasticsearch)	Sub-50ms queries, typo tolerance, easy self-host. Approved in Phase 0 Q7.	Architect (Phase 0 Part A)
MDR-008	Use Sentry + CloudWatch + UptimeRobot for observability	Three signals: errors, metrics, uptime. Cost-effective. Approved in Phase 0 Q9.	Architect (Phase 0 Part A)
MDR-009	80% line / 70% branch coverage, E2E mandatory for critical paths	Industry standard. Enforced in CI. Approved in Phase 0 Q10.	Architect (Phase 0 Part A)
MDR-010	Preserve product URL as /game-source-code/{slug} (not /products/{slug})	Source code confirmed this is the actual URL. Laravel already has 301 redirects from /products/{slug}. Preserve for SEO.	Architect (Phase 0 Part B)
MDR-011	Use single seller_earning formula: (price - (seller_fee + discount)) + extended_amount + addon_services_amount	Source code has TWO inconsistent formulas. This one matches createOrder() and is more accurate. Document and use consistently.	Architect (Phase 0 Part B)
MDR-012	Move all async work (email, file upload, notifications) to BullMQ	Source code has zero queue usage — all synchronous. NestJS must use BullMQ for p95 latency target.	Architect (Phase 0 Part B)
MDR-013	Sanitize blog post body with DOMPurify on render	Source code uses raw {!! \$post->body !!} — XSS risk. NestJS + React should sanitize.	Architect (Phase 0 Part B)
MDR-014	Flag synthetic reviews + sales counts in new schema (is_synthetic, is_sales_count_verified)	Source code confirmed fake data via Artisan commands. Preserve user-facing data but flag for internal reporting.	Architect (Phase 0 Part B)
MDR-015	Implement proper impersonation with JWT impersonated_by claim + audit log	Source code uses Auth::login() — no audit trail. NestJS should use proper impersonation token.	Architect (Phase 0 Part B)
MDR-016	Replace PayPal IPN with Stripe Checkout post-cutover	PayPal IPN deprecated. Stripe Checkout is modern, has webhooks with signature verification.	Architect (Phase 0 Part B)

MDR #	Decision	Rationale	Approved By
MDR-017	Use separate Razorpay webhook_secret (not key_secret) for webhook verification	Source code uses key_secret for both. Separate secret prevents webhook forgery if API key compromised.	Architect (Phase 0 Part B)
MDR-018	Split general_settings into 7 typed tables (SiteConfig, EmailConfig, SmsConfig, PushConfig, StorageConfig, SeoConfig, SocialiteConfig)	Source code has 50+ fields in one row. Splitting improves type safety and Prisma model clarity.	Architect (Phase 0 Part B)
MDR-019	Add rate limiting on ALL auth endpoints (login, register, password reset, contact form)	Source code only throttles login. NestJS should use @nestjs/throttler everywhere.	Architect (Phase 0 Part B)
MDR-020	Preserve cart as per-item model (do NOT introduce parent Cart entity)	Source code confirmed intentional design. Refactoring would change business logic. Migrate as-is.	Architect (Phase 0 Part B)

End of Phase 0 Discovery Document — Part B

Part B is now complete. Combined with Part A, this constitutes the full Phase 0 Discovery Document. Next step: approve Phase 0 (Part A + Part B), rotate exposed payment credentials, and proceed to Phase 1 (Foundation) — monorepo scaffolding, design system, auth module.